Algorithmen und Datenstrukturen
Übung 5 – Container

Aufgabe 1: Theorie

a) Templates ermöglichen es, Code zu schreiben, der beliebige Typen zulässt. Allerdings sind Templates selbst recht komplex. Dazu das folgende Beispiel.

Der Syntax für die Definition einer Template-Membermethode außerhalb der zugehörigen Template wäre (als Beispiel):

```
// In der .hpp-Datei:
template <typename A, typename B, typename R>
    class X
{
    public: R method(A a, B b);
};


// In der .cpp-Datei:
template <typename A, typename B, typename R>
    R X<A, B>::method(A a, B b)
        { return (R)(a + b); }
```

Ja, man muss wirklich für jede Methode die `template`-Zeile wiederholen. Ja, das ist wirklich nicht schön. Definitionen außerhalb der Template haben aber ein noch viel schwerwiegenderes Problem. Versuchen Sie dazu, die Vorlage `X` aus der `main` (in einer anderen Übersetzungseinheit) zu verwenden.

Beantworten Sie anschließend folgende Fragen:

- Was geht schief? Warum? Wie äußert sich das Problem?
- Warum löst sich das Problem, wenn man die Methode innerhalb der Vorlage statt außerhalb davon definiert?

Hinweise:

- Wie sieht der Maschinencode für die obige Methode aus?
- Welcher Zusammenhang besteht zu `.obj`-Dateien?

b) Es seien folgende Codedateien gegeben:

```
1    // Datei: swap_template.hpp
2    #if !defined(INCLUDED__BLATT_11__SWAP_TEMPLATE_HPP)
3    #define INCLUDED__BLATT_11__SWAP_TEMPLATE_HPP
4    template <typename T>
5        T swap(const T& a, const T& b);
6    #endif
```

```
7    // Datei: swap_template_a.cpp
8    #include "swap_template.hpp"
9    #include <iostream>
10
11   template <typename T>
12       void swap(T& a, T& b) {
13           a = b;
14           b = 0; // intentionally broken
15       }
16
17   void run();
18
19   int main() {
20       int a = 1, b = 2;
21       swap(a, b);
22       std::cout << a << " " << b << std::endl;
23       run();
24   }
```

```
25   // Datei: swap_template_b.cpp
26   #include "swap_template.hpp"
27   #include <iostream>
28
29   template <typename T>
30       void swap(T& a, T& b) {
31           T temp(a);
32           a = b;
33           b = temp;
34       }
35
36   void run() {
37       int c = 3, d = 4;
38       swap(c, d);
39       std::cout << c << " " << d << std::endl;
40   }
```

Beantworten Sie folgende Fragen zum obigen Code:

- Welche Werte stehen nach den `swap`-Operationen in den Variablen `a` bis `d`?
- Warum kompiliert das Programm, obwohl `swap()` mehrfach definiert wurde?
- Was passiert, wenn man eine Definition von `swap()` weglässt?
- Welche Schlussfolgerung für Templates ergibt sich daraus?

c) Angenommen, eine Vorlage `X` wird in vielen `.cpp`-Dateien verwendet, aber jeweils mit den selben Typparametern. Was für ein Problem entsteht daraus? Wie kann die Compilerpipeline vorgehen, um dieses Problem zu lösen?

Hinweis: Die Ergebnisse der vorangehenden Aufgaben sind hier relevant.

d) Welches Verhalten erzeugt das nachstehende Codebeispiel? Warum?

```
1    std::list<int> l( { 1, 2, 3 } );
2    std::list<int>::iterator it(l.begin());

3    it++; it++; it++; it++; it++;
```

```
4    // std::cout << *it;

5    it--; it--; it--;
6    std::cout << *it;
```

Hinweis: Gibt es Unterschiede zwischen Debug- und Release-Build? Was passiert, wenn die Ausgabe in Zeile 4 ausgeführt wird?

e) Ändert sich das Verhalten von oben, wenn man statt `list` einen `vector` nutzt?

Hinweis: Hier ist ein Blick in die Doku nötig, Compiler und Verstand helfen nicht!

f) Was ist das Problem an folgendem Codeausschnitt? Wie äußert sich das Problem (speziell im Release-Build)? Warum ist das Problem schwer zu finden?

```
1    std::list<int> a( { 3, 5, 7, 9, 11 } );
2    std::list<int> b( { 2, 4, 6, 7, 13 } );

3    auto x(std::find(a.begin(), a.end(), 7));
4    b.erase(x);
```

Hinweis: Wie würde eine zeitoptimale Implementierung für `erase` aussehen?

Aufgabe 2: Praxis

a) Stellen Sie Ihren `int_vector` aus dem vorherigen Übungsblatt zu einer `template` um, so dass der `int_vector` Einträge von beliebigen Typen `T` enthalten kann. Sie sollen dazu nur minimal nötige Änderungen am Grundkonzept der Klasse vornemen (z. B. weiter mit `malloc` Speicher allokieren). Benennen Sie das Ergebnis aber als Vorlage `vector` statt `int_vector`.

Erweitern Sie Ihre `main`, um den `vector` zu testen.

Hinweis: Die Teilaufgabe zur Implementierung von Template-Methoden von oben ist für diese Aufgabe vermutlich hilfreich!

b) Prüfen Sie, ob Sie in Ihren selbst-geschriebenen `vector` auch `std::string`s einfügen können. Falls nein: Warum funktioniert es nicht? Korrigieren Sie Ihren `vector` mit geeigneten Mitteln nach, bleiben Sie aber weiterhin bei der Verwaltung von Speicher mittels `malloc`. Sie dürfen `new` höchstens zusätzlich verwenden.

c) Erweitern Sie Ihren `vector` um einen `vector_iterator`. Diese Klasse soll folgende Funktionen bieten:

- Zugriff auf einen aktuellen Eintrag mittels `*` sowie `->` nach Vorlesung.
- Weiterzählen und Zurückzählen mit `++` und `--`.

- Zugriff auf einen Array-Eintrag mittels `[offset]`, wobei `offset` relativ zum aktuellen Eintrag des Iterators gewertet wird.
- Abfrage von Iteratoren zum Anfang des vectors per `begin()` und für One-Past-End des `vector`s mittels `end()`.

Prüfen Sie in der `main`, ob der `vector_iterator` funktioniert.

d) Was für einen Iterator haben Sie hier produziert? Ordnen Sie den Iterator gemäß den Kategorien der Vorlesung ein.

e) Lösen Sie die Teilaufgabe 2c) in einer einzigen, einfachen Codezeile.

Hinweis: Hier braucht man die richtige Idee. Tricks (z. B. einfach allen Code in eine einzige Zeile zu schreiben) sind nicht zulässig.